

Week15 연습과제

⚙ Status	Not started
≡ Text	424-468

BERT

- 2018년 구글에서 발표한 언어 모델
- 트랜스포머 기반 양방향 인코더 사용 자연어 처리 모델

양방향 인코더

- 입력 시퀀스를 양쪽 방향에서 처리 → 이전/이후의 단어를 모두 참조

기존 언어 모델

- 문장을 좌→우로 순차적 학습 ⇒ 의미와 문맥 파악할때 이전 단어만 고려

⇒ BERT: 양방향으로 학습 → 기존 모델보다 정확하게 문맥 파악, 다양한 작업에서 높은 성능

BERT 특징

- 대규모 데이터로 사전학습 → 전이 학습에 주로 활용
- BERT의 일부/전체를 다른 작업에서 재사용 → 적은 양의 데이터로도 높은 정확도, 모델 학습 시간 단축
- 트랜스포머의 인코더 기반 → 학습 데이터의 양과 다양성을 고려한 높은 성능
 - 인코더: 입력 문장의 단어 임베딩 → 각 단어의 의미 벡터화 → 순차적 처리 : 문장 전체의 의미 추출

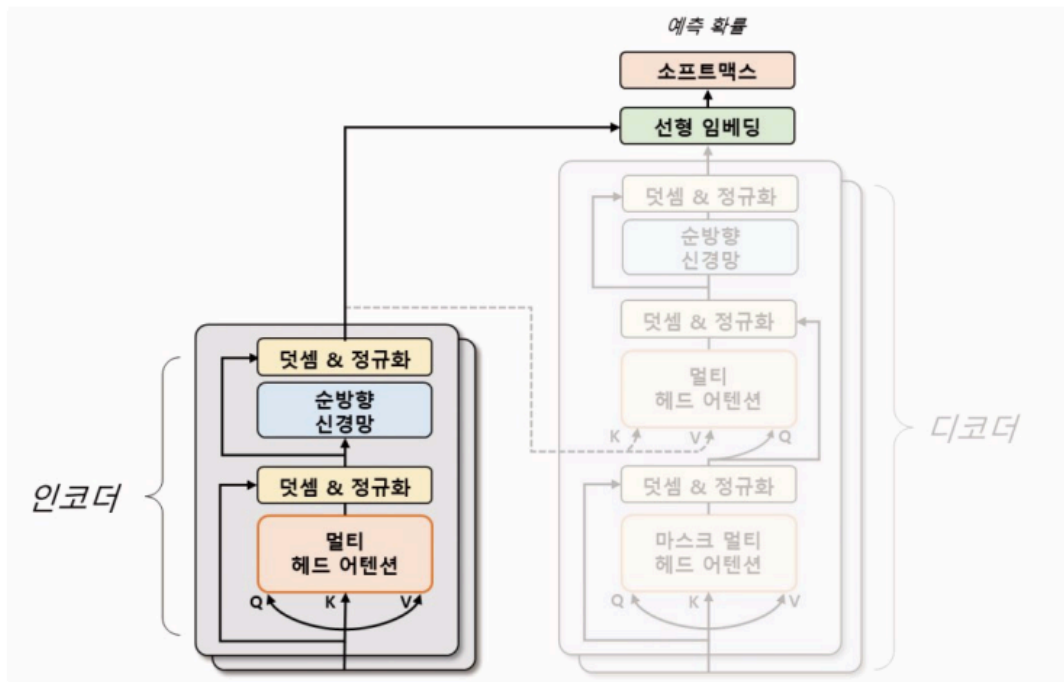


그림 7.13 트랜스포머와 BERT 모델 비교

사전 학습 방법

마스킹된 언어 모델 (Masked Language Modeling, MLM)

- 입력 문장에서 임의로 일부 단어 마스킹, 해당 단어 예측
- ex) 'I'm learning PyTorch' → 'learning' 마스킹 → 'I'm [MASK] PyTorch'
 - BERT: [MASK]에 들어갈 단어 예측(양방향 문맥 정보 참고)

다음 문장 예측 (Next Sentence Prediction, NSP)

- 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부 판단
- ex) 'I'm learning PyTorch', 'PyTorch is a machine learning library'
 - BERT: 두 문장이 연속적인 관계인지 아닌지 예측
 - 두 문장 간의 관계 학습, 문장 간의 의미적 유사성 파악
- 입력 문장에 특수한 토큰들을 추가 → 모델이 학습/추론하는 과정에서 필요한 정보 제공
 - [CLS]
 - 입력 문장의 시작 부분에 추가
 - 문장 분류 작업을 위한 정보 제공

- [SEP]
 - 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용
- [MASK]
 - 입력 문장 내에서 임의로 선택된 단어 가리킴
 - 주어진 문장에서 일부 단어 가린 후 모델의 학습과 예측에 활용

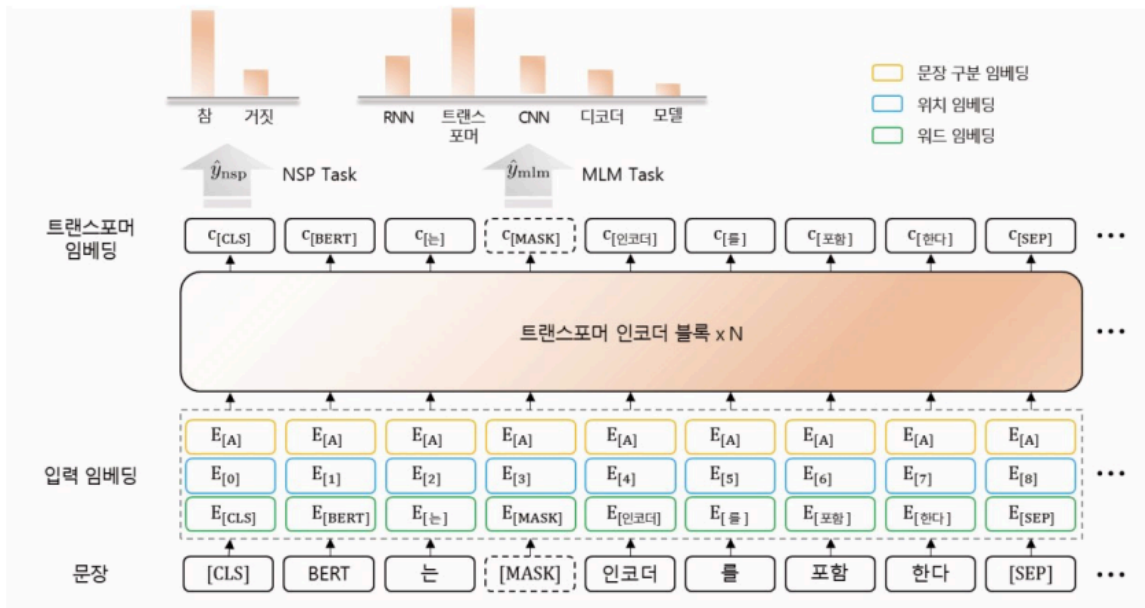


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

- 입력 문장: 'BERT는 트랜스포머 인코더를 포함한다'
- 텍스트 토큰 중 15%에 [MASK] 토큰 적용
 - 이중 80%는 [MASK] 토큰으로 대체
 - 10%는 어휘 사전에 존재하는 무작위 단어로 변경
 - 나머지 10% 실제 토큰 그대로 사용
- 사전 학습 후, 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용
 - 문장 분류, 감성 분석, 질문 응답, 기계 번역...
 - 미세 조정: 해당 작업에 맞는 계층 추가, 손실함수 정의
- 분류 문제 [CLS] 토큰 벡터 사용, 예측 필요한 경우 모든 토큰 벡터 사용

BART

- BERT 인코더 + GPT 디코더 결합 → Seq2Seq 구조 → 노이즈 제거 오토인코더로 사전 학습
- 오토인코더 사전 학습 방법
 - 입력 데이터에 잡음 추가
 - 잡음이 없는 원본 데이터를 복원하도록 학습
- 사전학습시 BERT의 인코더와 GPT의 디코더가 학습하는 방법을 일반화 → 학습

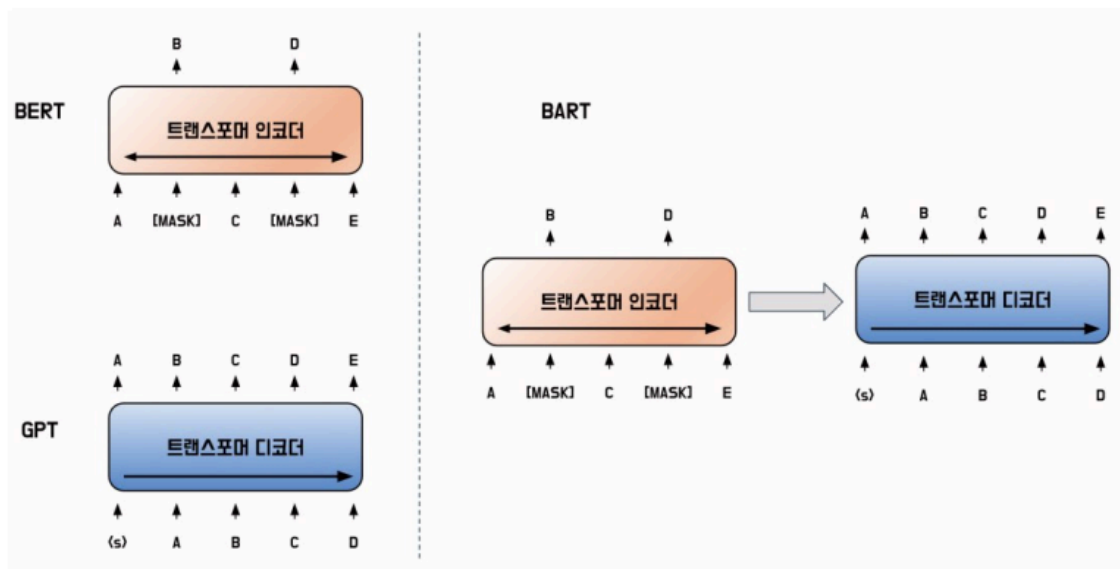


그림 7.15 BART 사전 학습 시각화

BART 사전 학습

- 노이즈 제거 오토인코더 기반
- 입력 문장에 임의의 노이즈 추가 → 원문 복원 학습
- 노이즈가 추가된 문장 → 인코더 입력
- 원본 문장 → 디코더의 target으로 사용
- 디코더가 원본 문장을 생성하도록 학습

노이즈 기법

- 단어 마스킹
- 단어 삭제

- 문장 순서 섞기
- 입력 문장에 제약 없이 적용 가능

장점

- 문장 구조와 의미를 유지한 채 다양한 변형 학습 가능
- 양방향 문맥 이해 가능 (GPT 단점 보완)
- 문장 생성 능력 우수 (BERT 단점 보완)

모델 구성 차이

- BERT: 인코더만 사용
- BART: 인코더 + 디코더 모두 사용

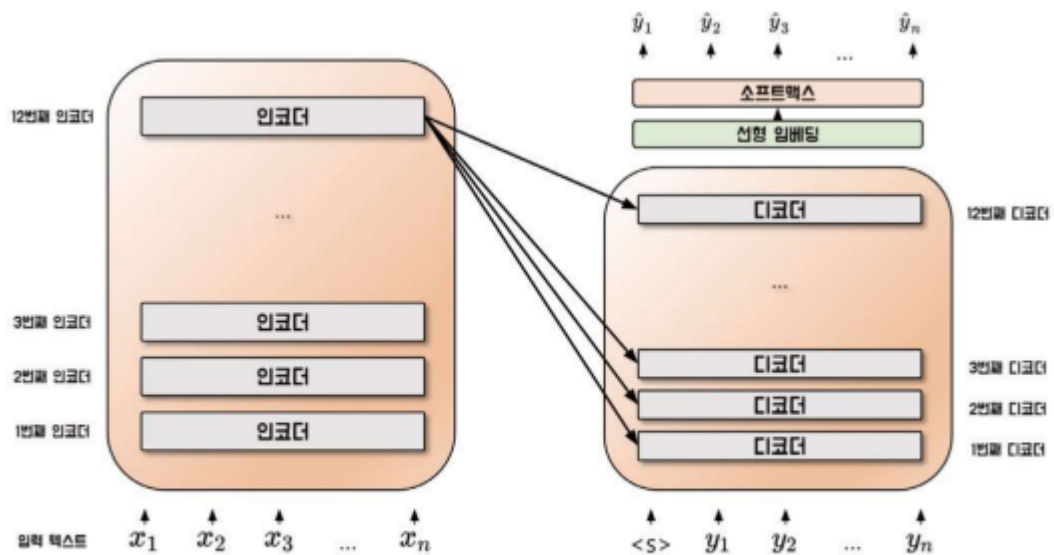


그림 7.16 BART 모델 구조

어텐션 연산 방식 차이

- 트랜스포머 : 인코더의 모든 계층 ↔ 디코더의 모든 계층 사이에서 어텐션 수행
- BART: 인코더의 마지막 계층과 디코더의 각 계층 사이에서만 어텐션 수행

BART 인코더

- 입력 문장의 각 단어를 임베딩
- 여러 인코더 계층을 거쳐 처리
- 마지막 계층 - 입력 문장 전체 의미를 가장 잘 반영한 벡터 생성

BART 디코더

- 인코더 마지막 계층에서 생성된 벡터 참고
- 이전 디코더 계층의 출력 정보와 함께 사용
- 각 계층에서 출력 문장을 점진적 생성

장점

- 어텐션 연산 범위 제한
- 정보 전달 과정 최적화
- 메모리 사용량 감소

사전학습

노이즈 기법

- BERT의 MLM 기반 토큰 마스킹 포함
- 추가 노이즈 기법
 - 토큰 삭제, 문장 교환, 문서 회전, 텍스트 채우기
- 다양한 형태의 문장 변형 학습 가능
- 입력 문장의 의미 복원 능력 강화

원본 문장	노이즈 기법
	토큰 마스킹
	그는 _ 발을 옮겼다. _ 아내는 멀어지는 그를 _있었다. 그녀는 눈물을 _ 그도 _였다.
	토큰 삭제
	그는 발을 옮겼다. 아내는 멀어지는 그를 있었다. 그녀는 눈물을 , 그도 였다.
	문장 순열
	그녀는 눈물을 흘렸다. 그는 정문으로 발을 옮겼다. 그의 아내는 멀어지는 그를 바라보고 있었다. 그도 마찬가지로였다.
	문서 회전
	그를 바라보고 있었다. 그녀는 눈물을 흘렸다. 그도 마찬가지로였다. 그는 정문으로 발을 옮겼다. 아내는 멀어지는
	텍스트 채우기
	그는 옮겼다. 아내는 멀어지는 그를 , 그녀는 눈물을 흘렸다. 그도 ,

그림 7.17 BART 노이즈 기법

토큰 마스킹 (Token Masking)

- BERT의 MLM과 동일한 기법
- 입력 문장의 일부 토큰을 마스크 토큰으로 치환
- 효과
 - 문장 내에서 중요한 단어 학습
 - 문맥 이해 능력 향상
 - 핵심 정보 추출 능력 강화

토큰 삭제 (Token Deletion)

- 입력 문장의 일부 토큰을 치환하지 않고 완전히 삭제
- 삭제된 위치와 토큰을 모두 예측해야 함
- 효과
 - 불필요하거나 중요하지 않은 정보 자동 필터링
 - 학습 및 예측 시간 감소
 - 모델 일반화 성능 향상

문장 순열 (Sentence Permutation)

- 마침표 기준으로 문장 분리
- 문장 순서를 임의로 섞음
- 모델은 원래 문장 순서를 복원해야 함
- 효과
 - 문장 간 논리적 연결 관계 학습
 - 문장 순서가 바뀌어도 안정적으로 처리 가능

문서 회전 (Document Rotation)

- 임의의 토큰을 문서 시작 지점으로 설정
- 문장 순서는 그대로 유지
- 모델은 원래 문서의 시작 토큰을 예측해야 함
- 효과
 - 문서의 시작점 인식 능력 향상

텍스트 채우기 (Text Infilling)

- 여러 토큰을 하나의 연속 구간으로 묶음
- 일부 구간을 마스크 토큰으로 대체
- 구간 길이
 - 0 이상 임의의 값
 - 일반적으로 포아송 분포 사용
 - 람다 = 3으로 설정
- 구간 길이=0
 - 해당 위치에 변환된 토큰 없음
- 학습 방식
 - 연속된 마스크 토큰 복구
 - 마스크되지 않은 토큰과 구분 필요
- 효과
 - 누락된 단어 예측 유도
 - 더 많은 문맥 정보 활용
 - 입력 문장 이해 능력 강화

미세 조정 방법

- 인코더와 디코더에 다른 문장 구조로 입력

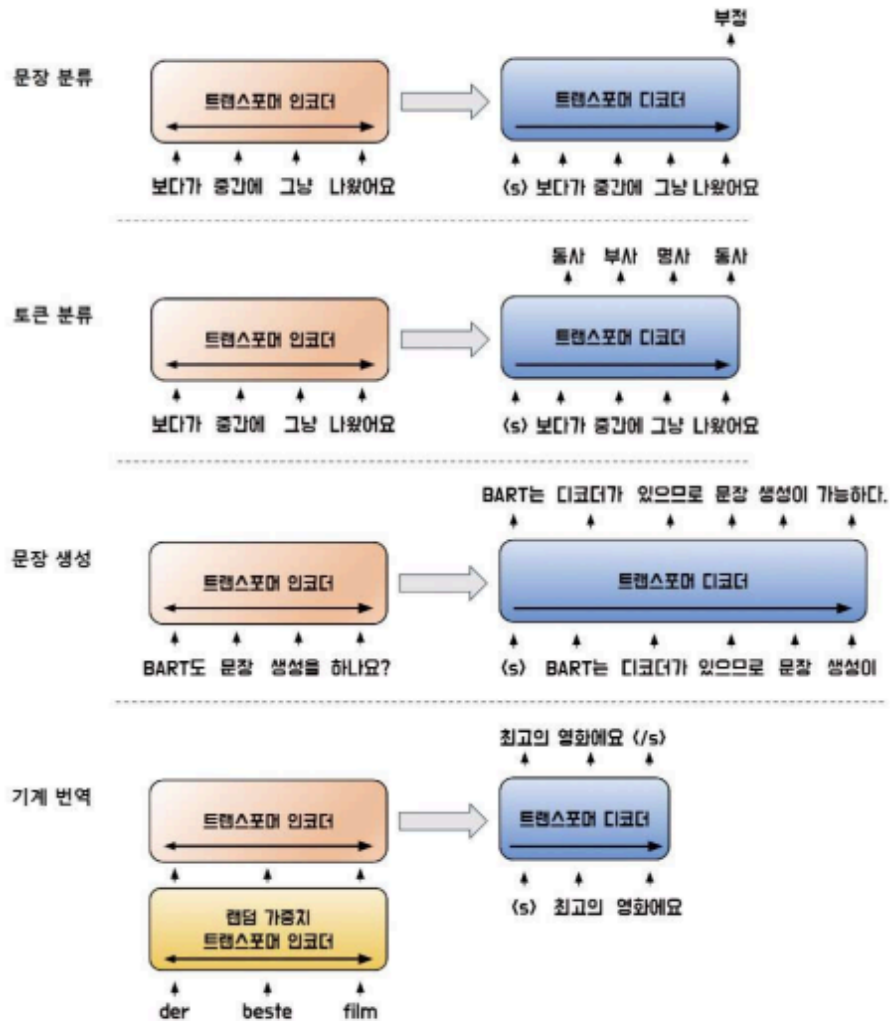


그림 7.18 BART 미세 조정 구조

문장 분류 작업

- 입력 문장: 인코더, 디코더 동일
- 디코더의 마지막 토큰 은닉 상태 - 선형 분류기의 입력값으로 사용
 - BERT의 CLS토큰과 유사, but 전체 입력과 어텐션 연산이 적용된 은닉 상태 사용

토큰 분류 작업

- 입력 문장: 인코더, 디코더에 동일하게 입력
- 디코더의 각 시점별 마지막 은닉 상태 - 토큰 분류기의 입력값으로 사용

BART 특징

- 트랜스포머 디코더 사용 → BERT가 해결하지 못한 문장 생성 작업 수행 가능

- 추상적 질의 응답, 문장 요약
- BART의 사전 학습 방식과 유사한 작업에서 뛰어난 성능

BART 모델 학습/평가

루지(Recall-Oriented Understudy for Gisting Evaluation, ROUGE) 점수

- 생성된 요약문과 정답 요약문의 유사도 - 토큰의 N-gram 정밀도와 재현율 이용

정답 문장 유니그램	[대한민국, 은, 16, 강, 에, 진출, 했다]
생성된 문장 유니그램	[대한민국, 은, 8, 강, 에, 진출, 하지, 못, 했다]
ROUGE-1 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$
ROUGE-1 정밀도	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$

정답 문장 바이그램	[대한민국은, 은 16, 16강, 강에, 에 진출, 진출했다]
생성된 문장 바이그램	[대한민국은, 은 8, 8강, 강에, 에 진출, 진출하지, 하지 못, 못 했다]
ROUGE-2 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$

그림 7.19 루지 점수 계산 방법

ROUGE-L

- 생성된 요약문과 정답 요약문 사이에서 최장 공통부분 수열 기반 통계 방식
- 두 문장 사이에 공통으로 존재하는 가장 긴 부분 문자열 찾는 문제
- 문장의 구조적 유사성 고려, 가장 길게 연속되는 N-gram 식별

ROUGE-LSUM

- ROUGE-L의 변형
- 텍스트 내의 개행 문자를 문장 경계로 인식
- 각 문장쌍에 대해 LCS 계산 → union-LCS 계산
 - union-LCS: 각 문장 쌍의 LCS 합친 것
 - 중복된 부분 제거 후 길이 계산
- 텍스트 생성 작업에서 요약의 정확성/완전성 모두 반영

ROUGE-W

- Weighted LCS-based
- 연속된 LCS에 가중치 부여하여 계산
- 공통부분 문자열의 길이 + 해당 부분 문자열 내의 단어에 가중치 부여해 평가

ELECTRA

Efficiently Learning an Encoder that Classifies Token Replacements Accurately

- 입력 마스킹하는 대신 생성자(Generator)와 판별자(Discriminator)사용해 사전 학습 수행
- 사전 학습 구조
 - 생성자
 - 실제 데이터와 유사한 토큰 생성
 - 입력 문장의 일부 토큰을 다른 토큰으로 대체
 - 판별자
 - 입력된 토큰이 실제/생성자가 만든 토큰인지 파별
- GAN과 유사성
 - 생성자와 판별자를 동시에 학습
 - 생성적 적대 신경망(GAN)과 유사한 학습 방식
- 장점
 - 더 효율적인 사전 학습 가능
 - 대규모 데이터셋에서 빠른 학습 속도

- 자연스러운 토큰 생성 → 다양한 자연어 생성 작업에 유리함
- 모델의 효율성
 - BERT 대비 매개변수 수 적음
 - 모델 크기 감소
 - 실행 속도 향상
 - 더 적은 메모리 사용

사전 학습 방법

- 생성자/판별자 모델 → 트랜스포머 인코더 구조 따름
- 생성자 모델: 입력 문장의 일부 토큰을 마스크 처리 → 처리된 토큰이 원래 어떤 토큰이었는지 예측
- 판별자: 각 입력 토큰이 원본 문장의 토큰인지 생성자 모델로 인해 바뀐 토큰인지 맞히며 학습

⇒ RTD(Replaced Token Detection)

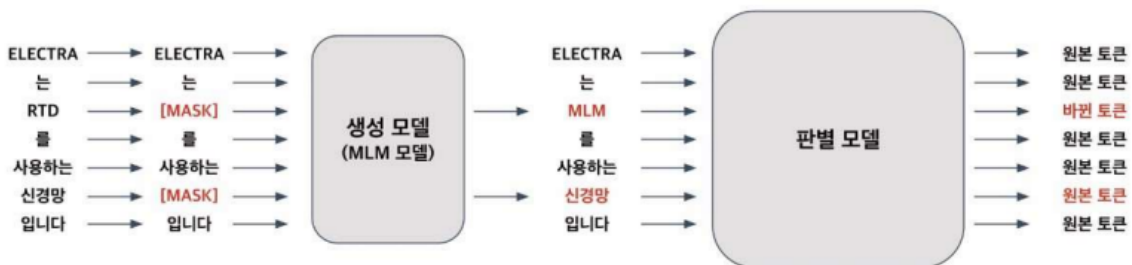


그림 7.20 ELECTRA 모델 학습 방법

GAN과 차이점

- 생성자 모델- 원래 토큰을 정확히 예측한 경우 원본 토큰으로 인식
- GAN: 생성 모델과 판별 모델 적대적 학습
 - ELECTRA: 마스킹된 언어 모델을 통해 학습
- GAN: 완전한 노이즈 벡터 입력받아 생성
 - ELECTRA: 일부 토큰이 마스크 처리된 텍스트를 입력으로 받음

특징

- 생성 모델/판별 모델 모두 트랜스포머 인코더 구조 따름
- 두 모델이 같은 개수의 계층으로 구성되어있으면 가중치 공유 가능 → 더 빠른 학습
- but 두모델이 가중치를 완전히 공유 → 생성 모델의 성능 너무 높음 → 판별 모델 학습 X
- ⇒ 생성 모델을 판별 모델의 크기를 1/2~1/4 크기로 바꿔 설정
- 모든 가중치를 공유하는 대신 임베딩 계층의 가중치만 공유
- ⇒ 판별 모델이 학습하기 적합한 생성모델 유지하면서도 가중치 공유하여 효율적으로 학습

T5

Text-to-Text Transfer Transformer

- 인코더-디코더 모델 구조
- GLUE, SuperGLUE, CNN/DM 등에서 SOTA(State-of-the-art)달성
- 다양한 자연어 처리 작업에서 높은 성능

기존 자연어 처리 모델

- 입력 문장을 벡터/행렬로 변환 → 출력 문장 생성
- 출력값이 클래스 or 입력값의 일부를 반환하는 형식

T5

- 입력과 출력을 모두 토큰 시퀀스로 처리 (Text-to-Text 구조)
- 입력과 출력의 형태를 자유롭게 다룰 수 있음
- 모델 구조상 유연성과 확장성이 뛰어나 → 새로운 자연어 처리 작업에도 쉽게 적용 가능

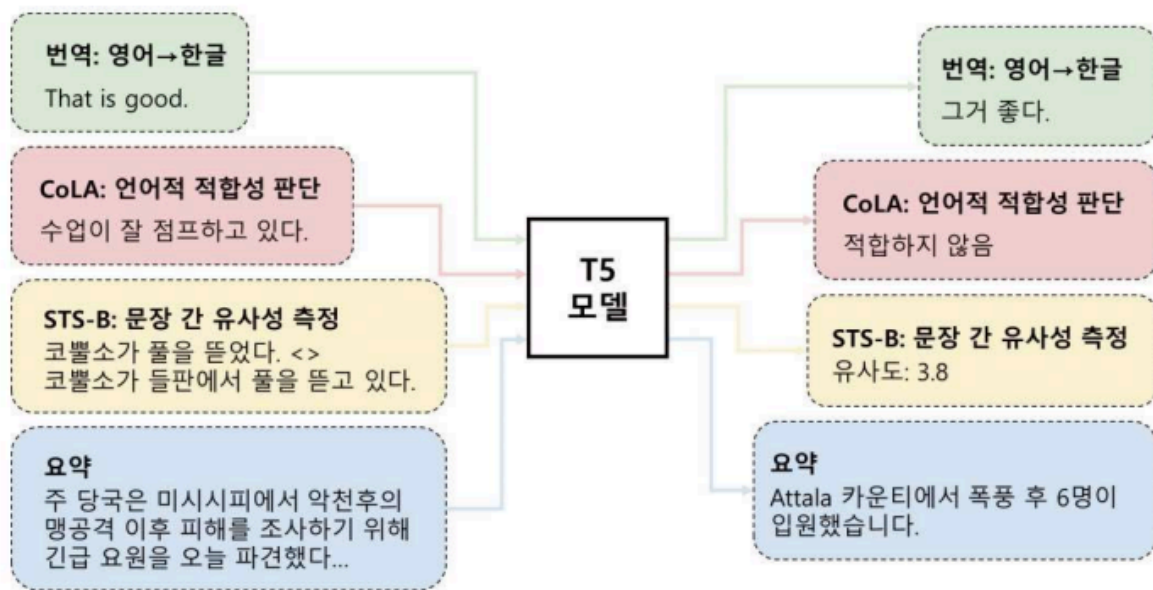


그림 7.21 T5 모델 학습 유형

- 입력과 출력이 모두 토큰 시퀀스 → 입력과 출력 간의 관계를 더욱 세밀하게 다룰 수 있음
- 사전 학습 수 미세 조정 단계에서 해당 작업의 데이터를 이용해 모델 조정 → 최적의 성능

CoLA 데이터 셋

- 문법적으로 허용 가능한 문장 구분

STS-B 데이터셋

- 문장 간 의미적 유사성 측정

C4 데이터셋

- 다양한 자연어 처리 작업 수행할 수 있게 사전학습

사전학습 방식

- 비지도 학습
- 입력 문장의 일부 구간 마스킹 → 입력 시퀀스 처리
- 출력 시퀀스: 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성
- 센티널 토큰: 문장마다 유일한 마스크 토큰

- 0부터 99개까지 100개의 기본값 사용
- 마스킹 토큰을 예측하는 것을 목적으로 사전학습됨
- 사전학습 완료 → 미세 조정 → 다양한 자연어 처리 작업 수행

미세 조정

- 지도학습 방식으로 학습